Algoritmos y Estructuras de Datos

Guía Práctica 5
Complejidad
Segundo Cuatrimestre 2026

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ejercicio 1. Probar que las siguientes afirmaciones son verdaderas utilizando las definiciones.

a) $n^2 - 4n - 2 \in O(n^2)$.

b) Para todo $k \in \mathbb{N}$ y toda función $f : \mathbb{N} \to \mathbb{N}$, si $f \in O(n^k)$, entonces $f \in O(n^{k+1})$.

c) Si $f : \mathbb{N} \to \mathbb{N}$ es tal que $f \in O(\log n)$, entonces $f \in O(n)$.

Solución

a) $n^2 - 4n - 2 \in O(n^2)$.
Queremos ver que existen c y n_0 tales que:
$n^2 - 4n - 2 \leq c.n^2$, $\forall n \geq n_0$
Asumo $n > 0$,
$\frac{n^2-4n-2}{n^2} \leq c$, $\forall n \geq n_0$
$1 - \frac{4}{n} - \frac{2}{n^2} \leq c$, $\forall n \geq n_0$
$1 - \frac{4}{n} - \frac{2}{n^2}$ es una función decreciente y siempre menor o igual a 1
Con $c = 1$, $n_0 \geq 0$ se cumple. Por lo tanto $n^2 - 4n - 2 \in O(n^2)$

b) Para todo $k \in \mathbb{N}$ y toda función $f : \mathbb{N} \to \mathbb{N}$, si $f \in O(n^k)$, entonces $f \in O(n^{k+1})$.
Queremos ver que existen c y n_0 tales que:
$n^k \leq c.n^{k+1}$, $\forall n \geq n_0$
Asumo $n > 0$,
$\frac{n^k}{n^{k+1}} \leq c$, $\forall n \geq n_0$
$\frac{1}{n} \leq c$, $\forall n \geq n_0$
$\frac{1}{n}$ es decreciente y vale máximo 1. Con $c = 1$ y $n_0 > 0$ vale.

Ejercicio 2. Determinar si cada una de las siguientes afirmaciones es verdadera o falsa. **Demuestre formalmente** su respuesta en cada caso.

a) $2^n \in O(1)$.

b) $n \in O(n!)$.

c) $n! \in O(n^n)$.

d) $2^n \in O(n!)$.

e) Para todo $i, j \in \mathbb{N}$ fijos,
con $f(n) = i \cdot n$, vale que $f \in O(j \cdot n)$.

f) Para todo $k \in \mathbb{N}$ fijo,
con $f(n) = 2^k$, vale que $f \in O(1)$.

g) $\log n \in O(n)$.

h) $n! \in O(2^n)$.

i) $n^5 + bn^3 \in \Theta(n^5) \iff b = 0$.

j) Para todo $k \in \mathbb{R}$ fijo,
con $f(n) = n^k log(n)$, vale que $f \in O(n^{k+1})$.

k) Para todo $f : \mathbb{N} \to \mathbb{N}$ vale que $f \in O(f)$.

Solución

a) $2^n \in O(1)$.

Falso.

$2^n \in O(1)$ sii $\exists n_0, c$ tq $\forall n \geq n_0, 2^n \leq c1$

2^n es una función creciente y no acotada, cómo tiende a infinito no se puede acotar con una constante c.

b) $n \in O(n!)$.

Verdadero.

$n \leq cn!$, $\forall n \geq n_0$

Reemplazo $n! = (n-1)!.n$

$n \leq c(n-1)!.n$, $\forall n \geq n_0$

$1 \leq c(n-1)!$, $\forall n \geq n_0$

Con $c = 1$ y cualquier n_0 vale.

c) $n! \in O(n^n)$.

Verdadero.

$n! \leq cn^n$, $\forall n \geq n_0$

$n! = n * n-1 * n-2 * .... * 3 * 2 * 1$

$n^n = n * n * n * .... * n * n * n$

Las dos expresiones tienen n términos, pero cada término de n^n es mayor o igual a cada término de $n!$ por lo que $n! \leq n^n$ (para todo n). Con por ejemplo $c = 1$ se cumple la definición.

d) $2^n \in O(n!)$.

Verdadero.

$n! = n * n-1 * n-2 * .... * 4 * 3 * 2 * 1$

$2^n = 2 * 2 * 2 * .... * 2 * 2 * 2 * 2$

Parecido al anterior, salvo el primer término, cada término i de 2^n es menor al término i de $n!$

Reescribiendo el término 4 de $n!$ como $2 * 2$:

$n! = n * n-1 * n-2 * .... * 2 * 3 * 2 * 2$ y se ve que cada término es mayor o igual a su respectivo término en 2^n cuando hay al menos 4 términos ($n_0 \geq 4$)

Entonces $2^n \leq n!$ y para $c = 1$ vale la definición.

e) Para todo $i, j \in \mathbb{N}$ fijos, con $f(n) = i \cdot n$, vale que $f \in O(j \cdot n)$.
Verdadero.

Queremos ver para cualquier par i, j naturales vale que existen n_0, c tales que:

$i * n \leq c * j * n$, $\forall n \geq n_0$

$i \leq c * j$ (ya no depende de n)

Paso dividiendo j (asumo distinto de 0)

$i/j \leq c$

i/j son constantes, así que se puede definir c en función de ellas.

Con, por ejemplo, $c = i/j + 1$ demostramos que se cumple la definición.

f) Para todo $k \in \mathbb{N}$ fijo, con $f(n) = 2^k$, vale que $f \in O(1)$.
Verdadero.

k es una constante, por lo que 2^k también.

g) $\log n \in O(n)$.

Verdadero.

$\log n \leq c.n$, $\forall n \geq n_0$

h) $n! \in O(2^n)$.

Por inducción voy a probar que $log(n) < n \forall n > 0$ (así vale la definición con $c = 1$).

Caso base, $n = 1$.

$log(1) = 0 < 1$

Hipótesis inductiva: $log(n+1) < n + 1$

log es una función creciente y para todo n mayor a 1 vale que $log(n+1) < log(2n)$ y por propiedad de logaritmos $log(2n) = log(2) + n$.

$log(n+1) < log(2) + log(n)$

Por el caso base, $log(n) < n$, y $log(2) < 1$. Por lo tanto $log(2) + log(n) < n + 1$.

$log(n+1) < log(2) + log(n) < 1 + n$. Queda probada la HI.

$log(n) < n, \forall n \geq 1$, por lo tanto $log(n) \in O(n)$.

Falso.

Supongo que es cierto. Entonces existen n_0 y c tales que:

$n! \leq c,2^n$, $\forall n \geq n_0$

Pruebo que existe un $m > n_0$ tal que esto no se cumple con $n \geq m$.

Sea $m = 2c$ (en caso de que $2c < n_0$, con $m = n_0 + 1$ la demostración funciona igual)

Con $m = 2c$ queda

$m! \leq c2^m$

$\frac{m!}{2^m} \leq c$

Reescribo $\frac{m!}{m}$ como $\frac{(m-1)!}{2^{(m-1)}} * \frac{m}{2}$ y queda:

$\frac{(m-1)!}{2^{(m-1)}} * \frac{m}{2} \leq c$

Reemplazo m por $2c$

$\frac{(m-1)!}{2^{(m-1)}} * \frac{2c}{2} = \frac{(m-1)!}{2^{(m-1)}} * c \leq c$

$\frac{(m-1)!}{2^{(m-1)}} \leq 1$

En el d) ya quedó demostrado que $2^n < n!$ a partir de $n = 4$ así que $\frac{m-1}{2^{(m-1)}}$ es mayor a 1

Por lo tanto, no existen n_0 y c que cumplan la definición.

i) $n^5 + bn^3 \in \Theta(n^5) \iff b = 0$.

Falso.

Contraejemplo:

Con $b = 1$, $n^5 + n^3 \in \Theta(n^5)$ por propiedad de la suma de funciones (nos quedamos con el término de mayor grado)

j) Para todo $k \in \mathbb{R}$ fijo, con $f(n) = n^k log(n)$, vale que $f \in O(n^{k+1})$.
Verdadero.

$n^k log(n) \leq c.n^{k+1}$

$log(n) \leq c.n$

Que ya sabemos que vale porque $log(n) \in O(n)$

k) Para todo $f : \mathbb{N} \to \mathbb{N}$ vale que $f \in O(f)$. **Verdadero.**

$f(n) \leq cf(n)$, con $c = 1$ vale para todo n.

Ejercicio 3. ¿Qué significa, intuitivamente, $O(f) \subseteq O(g)$? ¿Qué se puede concluir acerca del crecimiento de f y g cuando, simultáneamente, tenemos $O(f) \subseteq O(g)$ y $O(g) \subseteq O(f)$?

Ejercicio 4. Responder, justificando adecuadamente su respuesta:

a) Sean f y g el mejor y el peor caso de un algoritmo. ¿Es cierto entonces que $g \notin O(f)$?

b) Sean $g(n)$ y $h(n)$ la cantidad de operaciones que realizan los algoritmos G y H en función del tamaño de la entrada. Si G ejecuta la mitad de operaciones que H, ¿vale que $g \in \Theta(h)$?

c) Un algoritmo que toma un arreglo como input realiza $\Theta(n^2)$ operaciones cuando el arreglo tiene más de 100 elementos y $\Theta(1)$ operaciones cuando tiene 100 o menos. ¿Cuál es el mejor caso del algoritmo?

d) Si $f(n) < g(n), \forall n$, ¿es cierto que $f \notin \Omega(g)$?

e) Si la complejidad en el **peor caso** de un algoritmo es $\Omega(n)$, ¿es verdad que la complejidad de **mejor caso** no puede ser $O(1)$?

f) Si la complejidad temporal en el **peor caso** de un algoritmo pertenece a $O(n)$, entonces su complejidad temporal en el **mejor caso** también pertenece a $O(n^2)$.

Solución

a) Sean f y g el mejor y el peor caso de un algoritmo. ¿Es cierto entonces que $g \notin O(f)$?

Falso.

El mejor y peor caso de un algoritmo pueden coincidir. Por ejemplo, eliminar todas las apariciones de un elemento en una lista es siempre $\Theta(n)$. Por lo tanto $\Theta(f) = \Theta(g)$ (y en particular $g \in O(f)$).

b) Sean $g(n)$ y $h(n)$ la cantidad de operaciones que realizan los algoritmos G y H en función del tamaño de la entrada. Si G ejecuta la mitad de operaciones que H, ¿vale que $g \in \Theta(h)$?

Verdadero.

Por propiedades de los órdenes de complejidad, si $g \in \Theta(f)$ entonces para cualquier constante positiva c vale que $g.c \in \Theta(f)$.

c) Un algoritmo que toma un arreglo como input realiza $\Theta(n^2)$ operaciones cuando el arreglo tiene más de 100 elementos y $\Theta(1)$ operaciones cuando tiene 100 o menos. ¿Cuál es el mejor caso del algoritmo?

El mejor caso es $\Theta(n^2)$. El mejor caso no puede depender del tamaño del arreglo, así que no se puede considerar $\Theta(1)$ porque no se extiende a todos los tamaños (con $n \to \infty$).

d) Si $f(n) < g(n), \forall n$, ¿es cierto que $f \notin \Omega(g)$?

Falso.

Contraejemplo, $f(n) = n$ y $g(n) = n + 1$, $f < g$ pero $n \in \Omega(n + 1)(= \Omega(n))$.

e) Si la complejidad en el **peor caso** de un algoritmo es $\Omega(n)$, ¿es verdad que la complejidad de **mejor caso** no puede ser $O(1)$?

Falso.

La cota inferior del peor caso y la superior del mejor no se relacionan entre sí. Contraejemplo: en búsqueda en una lista desordenada, el mejor caso (que el primer elemento sea el que se está buscando) es O(1) y el peor (recorrer toda la lista) es $\Omega(n)$

f) Si la complejidad temporal en el **peor caso** de un algoritmo pertenece a $O(n)$, entonces su complejidad temporal en el **mejor caso** también pertenece a $O(n^2)$.

Verdadero.

El mejor caso va a ser siempre menor o igual al peor mejor caso $\in$ O(peor caso). Por transitividad, si el peor caso pertecenece a $O(n)$ también pertenece a $O(n^2)$ y por lo tanto también pasa con el mejor caso.

Ejercicio 5. Indique si las siguientes afirmaciones son verdaderas o falsas. En caso de ser **verdadera**, demuéstrelo formalmente. En caso de ser **falsa**, dé un contraejemplo claro que respalde su análisis. Sean $f, g, h : \mathbb{N} \to \mathbb{N}$

a) Si $O(f(n)) \cap \Omega(g(n)) = \emptyset$, entonces $O(g(n)) \cap \Omega(f(n)) = \emptyset$.

b) Si $f \in O(g)$, entonces $f \in \Theta(g) \cup \Theta(h)$ para cualquier función h.

c) Si $f \in O(g)$ y $h \in O(g)$, entonces $(f + h) \in O(g)$

d) $O(n^2) \cap \Omega(n) = \Theta(n^2)$

e) $\Theta(n) \cup \Theta(n \log n) = \Omega(n \log n) \cap O(n)$

f) $f \in O(g) \iff O(f) \subseteq O(g)$

g) Si $f \in \Omega(g)$, entonces $O(f) \cap \Omega(g) = O(g) \cap \Omega(f)$

h) Si $f(n) < g(n)$ para todo n, entonces $\Theta(f) \neq \Theta(g)$

i) Si $f \in O(g)$, entonces $f * g \in \Theta(g)$

Solución

a) Si $O(f(n)) \cap \Omega(g(n)) = \emptyset$, entonces $O(g(n)) \cap \Omega(f(n)) = \emptyset$.

Falso. Contraejemplo:

Con $f = 1$, y $g = n^2$, $O(f(n)) \cap \Omega(g(n)) = \emptyset$.

Pero $n \in O(n^2)$ y $n \in \Omega(1)$. Por lo tanto, $O(g(n)) \cap \Omega(f(n)) \neq \emptyset$

b) Si $f \in O(g)$, entonces $f \in \Theta(g) \cup \Theta(h)$ para cualquier función h.

Falso. $f \in O(g)$, no implica $f \in \Theta(g)$

Contraejemplo:

Con $f = n$, $g = n^2$ y $h = n^3$, $f \notin \Theta(g)$, $f \notin \Theta(h)$

c) Si $f \in O(g)$ y $h \in O(g)$, entonces $(f + h) \in O(g)$

Verdadero.

Como $f \in O(g)$ existen d y m_0 que cumplen la definición. Igual para h (b y k_0)

Con $c = d + b$ y $n_0 = max(m_0, k_0)$, se cumple que

$f(n) + g(n) \leq ch$, $\forall n \geq n_0$

(También se puede usar la propiedad de la suma)

d) $O(n^2) \cap \Omega(n) = \Theta(n^2)$

Falso.

$n \in O(n^2) \cap \Omega(n)$, pero $n \notin \Theta(n^2)$

e) $\Theta(n) \cup \Theta(n \log n) = \Omega(n \log n) \cap O(n)$

Falso.

$n \in \Theta(n) \cup \Theta(n \log n)$ pero $n \notin \Omega(n \log n) \cap O(n)$

$\Omega(n \log n) \cap O(n) = \emptyset$, porque $n \notin \Omega(nlog(n)$

f) $f \in O(g) \iff O(f) \subseteq O(g)$

Verdadero. Por la propiedad de transitividad.

Si $f \in O(g)$ y $h \in O(f)$ entonces $h \in O(g)$ (Entonces todos los elementos de $O(f)$ están contenidos en $O(g)$)

g) Si $f \in \Omega(g)$, entonces $O(f) \cap \Omega(g) = O(g) \cap \Omega(f)$

Falso. Contraejemplo:

$f = n^2$ y $g = 1$, $n \in O(n^2) \cap \Omega(1)$ pero $O(1) \cap \Omega(n^2) = \emptyset$

h) Si $f(n) < g(n)$ para todo n, entonces $\Theta(f) \neq \Theta(g)$

Falso. Contraejemplo:

Con $f = n$ y $g = n + 1$, $f < g$ para todo n. Pero como sumar una constante a una función no cambia la cota $\Theta(g) = \Theta(f) = \Theta(n)$

i) Si $f \in O(g)$, entonces $f * g \in \Theta(g)$

Falso. Contraejemplo:

Con $f = n$ y $g = n$, $f \in O(g)$ pero $n^2 \notin \Theta(g)$

Ejercicio 6. Determinar el orden de complejidad temporal de peor caso de los siguientes algoritmos, asumiendo que todas las operaciones sobre arreglos y matrices toman tiempo $O(1)$. La complejidad se debe calcular en función de una medida de los parámetros de entrada, por ejemplo, la cantidad de elementos en el caso de los arreglos y matrices y el valor en el caso de parámetros naturales.

a) SUMATORIA, que calcula la sumatoria de un arreglo de enteros:

```
proc sumatoria(array A)
  int i, total
  int total := 0

  for i := 0 to |A| - 1 do
    total := total + A[i]
  end for

  return total
end proc
```

b) SUMATORIALENTA, que calcula la sumatoria de n, definida como la suma de todos los enteros entre 1 y n, de forma poco eficiente:

```
proc sumatoriaLenta(nat n)
  int i, total
  int total := 0

  for i := 0 to n do
    for j := 0 to i do
      total := total + 1
        end for
  end for

  return total
end proc
```

c) PRODUCTOMAT, que dadas dos matrices A (de $p \times q$) y B (de $q \times r$) devuelve su producto AB (de $p \times r$):

```
proc productoMat(Matriz A, Matriz B)
  int fil, col, val colAFilB
  Matriz res(Filas(A), Columnas(B))

  for fil := 0 to Filas(A) - 1 do
    for col := 0 to Columnas(B) - 1 do
      val := 0
      for colAFilB := 0 to Columnas(A) - 1 do
        val := val + (A[fil][colAFilB] * B[colAFilB][col])
      end for
      res[fil][col] := val
    end for
  end for

  return res
end proc
```

d) INSERTIONSORT, que ordena un arreglo pasado como parámetro:

```
proc insertionSort(array A)
  int i, j, valor

  for i := 0 to |A| -1
    valor := A[i]
    j := i - 1

    while j >= 0 && a[j] > valor do
      A[j+1] := A[j]
      j := j - 1
    end while

    A[j + 1] := valor
  end for
end proc
```

e) BúsquedaBinaria, que determina si un elemento se encuentra en un arreglo, que debe estar ordenado:

```
proc busquedaBinaria(array A, elem valor)
  int izq := 0, der := Long(A) - 1

  while izq < der do
    int medio := (izq + der) / 2
  if valor < A[medio]
      der := medio
  else
    izq := medio
  end if
  end while

  return A[izq]
end proc
```

Solución

a) Sumatoria, que calcula la sumatoria de un arreglo de enteros:

```
1: function Sumatoria(arreglo A)
2:     int i, total; // O(1)
3:     total := 0; // O(1)
4:     for i := 0 ... Long(A) - 1 do // Θ(n)
5:         total := total + A[i]; // O(1)
6:     end for
7: end function
```

Defino $n = Long(A)$, cantidad de elementos del arreglo.

El ciclo se ejecuta siempre la misma cantidad de veces (no hay casos dónde se interrumpa antes de las n iteraciones)

Complejidad peor caso= $O(1) + O(1) + \sum_1^n O(1) = 2O(1) + \Theta(n) = \Theta(n)$

b) SumatoriaLenta, que calcula la sumatoria de n, definida como la suma de todos los enteros entre 1 y n, de forma poco eficiente:

```
1: function SumatoriaLenta(nat n)
2:     int i, total; // O(1)
3:     total := 0; // O(1)
4:     for i := 1 ... n do // Θ(n)
5:         for j := 1 ... i do // Θ(i)
6:             total := total + 1; // O(1)
7:         end for
8:     end for
9: end function
```

Los dos ciclos se ejecutan siempre n e i veces, la variable del ciclo de adentro depende de la variable i (que es distinto en cada iteración del ciclo exterior).

Complejidad peor caso= $O(1) + \sum_{i=1}^n \sum_{j=1}^i O(1)$

$\sum_{j=1}^i O(1) = i$

Complejidad peor caso= $O(1) + \sum_{i=1}^n \sum_{j=1}^i O(1) = O(1) + \sum_{i=1}^n i$

$\sum_1^n i = \frac{1}{2} i(i+1)$

Complejidad peor caso= $O(1) + \frac{i(i+1)}{2} = O(1) + \frac{i^2+i}{2} = \Theta(n^2)$

c) ProductoMat, que dadas dos matrices A (de $p \times q$) y B (de $q \times r$) devuelve su producto AB (de $p \times r$):

```
1: function ProductoMat(matriz A, matriz B)
2:     int fil, col, val, colAFilB; // O(1)
3:     matriz res(Filas(A), Columnas(B)); // Θ(pr)
4:     for fil := 0 ... Filas(A) - 1 do // Θ(p)
5:         for col := 0 ... Columnas(B) - 1 do // Θ(r)
6:             val := 0; // O(1)
```

```
7:              for colAFilB := 0 ... Columnas(A) - 1 do // Θ(q)
8:                  val := val + (A[fil][colAFilB] * B[colAFilB][col]); // O(1))
9:              end for
10:             res[fil][col] := val; // O(1)
11:         end for
12:     end for
13:     return res; // O(1)
14: end function
```

Los tres ciclos se ejecutan siempre la misma cantidad de veces (p, r y q veces).

Complejidad peor caso= $O(1) + \Theta(pr) + \sum_1^p [\sum_1^r (O(1) + \sum_1^q O(1) + O(1))] + O(1)$

Complejidad peor caso= $O(1) + \Theta(pr) + pqr = \Theta(pqr)$

d) InsertionSort, que ordena un arreglo pasado como parámetro:

```
1: function InsertionSort(arreglo A)
2:     int i, j, valor; // O(1)
3:     for i := 0 ... Long(A) - 1 do // Θ(n)
4:         valor := A[i]; // O(1)
5:         j := i - 1; // O(1)
6:         while j ≥ 0 ∧ a[j] > valor do // Θ(i − 1) = Θ(i)
7:             A[j+1] := A[j]; // O(1)
8:             j := j - 1; // O(1)
9:         end while
10:        A[j+1] := valor; // O(1)
11:    end for
12: end function
```

El ciclo externo siempre se ejecuta n veces (n= cantidad de elementos del arreglo). En cada iteración, el ciclo interno se ejecuta cómo mucho i veces pero puede detenerse antes si $A[j] \leq A[i]$. Cómo queremos analizar la complejidad en el peor caso, sólo vamos a considerar el caso en que $A[j] > A[i]$ siempre (o sea, el ciclo interno se ejecuta siempre la mayor cantidad de veces posible). Para que esto ocurra el arreglo de entrada tiene que estar en el órden opuesto al deseado (en orden decreciente si se quiere ordenar de manera creciente).

Complejidad peor caso= $O(1) + \sum_{i=1}^n [O(1) + \sum_{j=1}^i O(1) + O(1)]) = O(1) + \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$

e) BúsquedaBinaria, que determina si un elemento se encuentra en un arreglo, que debe estar ordenado:

```
1: function BúsquedaBinaria(arreglo A, elem valor)
2:     int izq := 0, der := Long(A) - 1; // O(1)
3:     while izq < der do // O(log(n))
4:         int medio := (izq + der) / 2; // O(1)
5:         if valor < A[medio] then // O(1)
6:             der := medio; // O(1)
7:         else
8:             izq := medio; // O(1)
9:         end if
10:    end while
11:    return A[izq] = valor; // O(1)
12: end function
```

En este caso, la variable de control del ciclo no disminuye de 1 en 1 sino que se divide por 2. Empezando con $izq = 0$ y $der = n$ se pueden realizar como mucho $log(n)$ iteraciones (en particular es log en base 2 pero el cambio de base de logaritmo no afecta la cota de complejidad).

Complejidad peor caso= $O(1) + \sum_{i \in \{n, \frac{n}{2}, \frac{n}{4}, \ldots\}} O(1) + O(1) = \Theta(log(n))$

Ejercicio 7. Determinar y explicar con palabras la complejidad del peor caso de los siguientes algoritmos.

a)
```
proc sumaCubos(int n)
    i := 1
    res := 0
```

```
        while i < n^2 do
            fact := 1
            j := 0
            while j < 3 do
                fact := fact * i
                j := j + 1
            end while
            res := res + fact
            i := i + 1
        end while
    end proc

    proc sumaPotenciasDe2(int n)
        i := 1
        res := 0
        while i < n do
            res := res + 1
            i := i * 2
        end while
    end proc
```

b)
```
    proc sumaRara(array A)
        i := 0
        res := 0
        while i < |A| do
            j := 0
            while j < i do
                res := res + A[j]
                j = j + 1
            end while
            i := i + 1
        end while
    end proc
```

Ejercicio 8. Para cada una de las siguientes afirmaciones, decida si son verdaderas o falsas y demuestre formalmente su decisión.

a) $n + m \in O(nm)$.

b) $n + m^5 \in O(m^5)$.

c) $nm \in O(n + m)$.

d) $m^5 \in O(n + m^5)$.

Solución

a) $n + m \in O(nm)$.

Verdadero.

Queremos ver que existen C, n_0 y m_0 tales que: $n + m \leq Cnm$, $\forall n \geq n_0, m \geq m_0$

$\frac{n+m}{nm} \leq C$, $\forall n \geq n_0, m \geq m_0$

$\frac{n}{nm} + \frac{m}{nm} \leq C$, $\forall n \geq n_0, m \geq m_0$

$\frac{1}{m} + \frac{1}{m} \leq C$, $\forall n \geq n_0, m \geq m_0$

$\frac{1}{n}$ y $\frac{1}{m}$ son funciones decrecientes y acotadas con máximo igual a 1. Entonces con C=2 y $m_0 = 1 = n_0$, vale que:

$\frac{1}{m} + \frac{1}{m} \leq 1 + 1 \leq 2$, $\forall n \geq 1, m \geq 1$

b) $n + m^5 \in O(m^5)$.

Falso.

$n + m^5$ depende de dos variables y m^5 de sólo una de ellas. No se puede acotar a una variable con otra si son independientes entre sí.

Supongo que existen C, n_0 y m_0 tales que:

$n + m^5 \leq Cm^5$, $\forall n \geq n_0, m \geq m_0$ $n \leq Cm^5 - m^5$ $n \leq m^5(C-1)$

n es una función creciente y no acotada. Para cualquier m_1, no vale la definición para todos los $n > m^5(C-1)$

c) $nm \in O(n+m)$.

Falso.

Queremos ver que existen C, n_0 y m_0 tales que:

$nm \leq C(n+m)$, $\forall n \geq n_0, m \geq m_0$

$1 \leq \frac{Cm}{nm} + \frac{Cn}{nm}$, $\forall n \geq n_0, m \geq m_0$

$1 \leq \frac{C}{n} + \frac{C}{m}$, $\forall n \geq n_0, m \geq m_0$

Tanto $\frac{C}{n}$ como $\frac{C}{m}$ son funciones decrecientes que convergen al 0.

Por definición de límite, para $\epsilon = 0{,}25$ existe n_0 (y análogamente un m_0) tal que $|\frac{C}{n} - 0| < \epsilon$.

Con $n \geq n_0$ y $m \geq m_0$ entonces vale que:

$\frac{C}{n} + \frac{C}{m} < 2\epsilon = 0{,}5$, por lo que

$1 \not\leq \frac{C}{n} + \frac{C}{m}$, $\forall n \geq n_0, m \geq m_0$

Por lo tanto $nm \notin O(n+m)$.

d) $m^5 \in O(n + m^5)$.

Verdadero.

Queremos ver que existen C, n_0 y m_0 tales que:

$m^5 \leq C(n + m^5)$, $\forall n \geq n_0, m \geq m_0$

Con $C = 1$, queda:

$m^5 \leq n + m^5$

$0 \leq n$, que vale para cualquier n y m

Ejercicio 9. Sean $f, g : \mathbb{N} \to \mathbb{N}$, y supongamos que está definido el límite $\lim_{n\to+\infty} \frac{f(n)}{g(n)} = \ell \in \mathbb{R}_{\geq 0} \cup \{+\infty\}$. Probar que:

a) $0 < \ell < +\infty$ si y sólo si $f \in \Theta(g)$.
b) $\ell = +\infty$ si y sólo si $f \in \Omega(g)$ y $f \notin O(g)$.
c) $\ell = 0$ si y sólo si $f \in O(g)$ y $f \notin \Omega(g)$.

Recordar las definiciones de límite:

- $\lim_{n\to+\infty} a_n = \ell \in \mathbb{R}$ si $\forall\, \varepsilon > 0.\, \exists n_0 \in \mathbb{N}.\, \forall n > n_0.\, |a_n - \ell| < \varepsilon$.
- $\lim_{n\to+\infty} a_n = +\infty$ si $\forall\, M > 0.\, \exists n_0 \in \mathbb{N}.\, \forall n > n_0.\, a_n > M$.

Solución

a) $0 < \ell < +\infty$ si y sólo si $f \in \Theta(g)$.

$\implies$)

Si el límite de f/g existe y es finito entonces:

$\forall\, \varepsilon > 0.\, \exists n_0 \in \mathbb{N}.\, \forall n > n_0.\, |f(n)/g(n) - \ell| < \varepsilon$.

Saco el módulo (la demostración funciona igual para ambos casos):

$f(n)/g(n) - \ell < \varepsilon$.

$f(n) < (\varepsilon + \ell)g(n)$

Tomando $c = (\varepsilon + \ell)$ y el mismo n_0 podemos decir que $f \in O(g)$.

Cómo el límite de f/g existe también existe el de g/f (podemos suponer que el límite de f no es 0, porque partimos de $0 < \ell$) así que también podemos llegar a que $g \in O(f)$.

Cómo $g \in O(f)$ y $f \in O(g)$ entonces $f \in \Theta(g)$ (y viceversa).

$\Longleftarrow$)

Si $f \in \Theta(g)$, entonces $f \in O(g)$ y $f \in \Omega(g)$.

De $f \in O(g)$ sabemos que existen n_1 y c tales que:

$f/g \leq c$, $\forall n \geq n_1$

Por lo tanto f/g está acotada superiormente a partir de $n \geq n_1$.

Como $f \in \Omega(g)$ entonces existen n_2 y d tales que:

$f > dg$,$\forall n \geq n_2$ $f/g > d$

Entonces f/g también está acotada inferiormente.

Como f/g está acotada tanto superior como inferiormente a partir de cierto n_0 y sabemos que el límite existe, este límite tiene que ser finito.

Este límite no puede ser 0 (porque por $f \in \Omega(g)$ existe $d > 0$ tal que $f/g > d$ así que con $\varepsilon < d$ no puede cumplirse la definición de límite). Por lo tanto el límite es finito y mayor a 0.

b) $\ell = +\infty$ si y sólo si $f \in \Omega(g)$ y $f \notin O(g)$.

$\Longrightarrow$)

Si el límite es más infinito entonces $\forall\, M > 0. \exists n_0 \in \mathbb{N}. \forall n > n_0. f/g > M$.

Entonces $f > Mg$, $n > n_0$, por lo tanto $f \in \Omega(g)$

Por otro lado vemos que no pueden existir c y n_1 tales que:

$f < cg$, $\forall n \geq n_0$ $f/g < c$

Pero como el límite tiende a infinito para cualquier constante (en particular c) existe n_0 tal $f/g > c$, $\forall n \geq n_0$. Por lo tanto $f \notin O(g)$.

$\Longleftarrow$)

Como $f \notin O(g)$ no existe c tal que:

$f/g < c$ con n tendiendo a infinito

f/g no está acotada inferiormente. Cómo el límite existe, entonces tiene que converger al infinito.

c) $\ell = 0$ si y sólo si $f \in O(g)$ y $f \notin \Omega(g)$.

$\Longrightarrow$)

Cómo el límite es 0:

$\forall\, \varepsilon > 0. \exists n_0 \in \mathbb{N}. \forall n > n_0. |f(n)/g(n)| < \varepsilon$.

$f/g < \varepsilon$, $\forall n > n_0$

$f < \varepsilon g$, $\forall n > n_0$.

Entonces $f \in O(g)$.

Cómo $f < g\varepsilon$ para cualquier ε (a partir de cierto n) no pueden existir c y n_0 tales que $f > gc$ con $n \geq n_0$. Por lo tanto $f \notin \Omega(g)$

$\Longleftarrow$)

De $f \in O(g)$ tenemos que f/g es una función acotada superiormente($\leq c$) a partir de cierto n_0:

$f \leq gc$, $\forall n \geq n_0$

$f/g \leq c$, $\forall n \geq n_0$

Cómo $f \notin \Omega(g)$ no existen n_0 y d tales que:

$f \geq dg$,$\forall n \geq n_0$

$f/g \geq d$, $\forall n \geq n_0$

Cómo no existe ningún d, esto implica que $\forall \varepsilon > 0$ existe n_0 tal que:

$f/g < \varepsilon$,$\forall n \geq n_0$.

Por lo tanto f/g converge a 0.